

Java

```
abstract class Abstraction{

    abstract void display(); // incomplete method
    abstract void fiit();

    void showDetails(){
        System.out.println("Show details");
    }
}

class Child extends Abstraction{

    void display(){
        System.out.println("Display method from child class");
    }
    void fiit(){
        System.out.println("Fiit ");
    }
}
// this and super

class Student{

    String name;

    Student(String name){
        this.name = name;
        System.out.println("Student COnstructor called");
        System.out.println(this.name);
    }
}

class PT extends Student{

    PT(){
        super("Harish"); // parent constructor
        System.out.println("PT constructor called");
    }
}

public class Day4a {

    public static void main(String[] args) {

        PT obj = new PT();
    }
}
```

```
        System.out.println("Abstraction");
    }
}
```

```
interface Student{
    void display();
}

interface Teacher extends Student{

}

// same => extends
// different => implement

class PET implements Student{
    public void display(){
        System.out.println("Hello there");
    }
}

class Multiple implements Student , Teacher{
    public void display(){
        System.out.println("Display method");
    }
}

class WoH {

}

class Hybrid extends WoH implements Student{

    public void display(){
        System.out.println("Hello world");
    }
}

final class FinalKeyword{

    final int data = 10; // field

    final void display(){
```

```

        System.out.println("Display method from finalKeyword class");
    }
}

class Subclass extends FinalKeyword{

    void display(){
        System.out.println(10);
    }
}

// field => don't allow to modify values
// method => don't allow to override in subclass
// class => don't allow class to inherit

public class Interface {

    public static void main(String[] args) {

        FinalKeyword fkey = new FinalKeyword();
        // fkey.data = 40;
        System.out.println(fkey.data);

        fkey.display();

        PET obj = new PET();

        obj.display();

    }

}

```

```

package harini;

public class Harini {

    // private => within the class
    // default => same class, same package
    // protected => same class, same package , accessbile but with inheritance
    // public  => access everywhere

    // private int salary = 50000;

```

```
// int amount = 10000;

protected String name = "Malathi";

public int marks = 100;

}
```

```
package harini;

public class Manisha {

    public static void main(String[] args) {

        Harini obj = new Harini();

        // System.out.println(obj.salary);
        // System.out.println(obj.amount);
        System.out.println(obj.name);
        System.out.println(obj.marks);
    }

}
```

```
package subhiksha;

import harini.Harini;

public class Subhiksha extends Harini {

    public static void main(String[] args) {

        Subhiksha obj = new Subhiksha();

        System.out.println(obj.name);

        System.out.println(obj.marks);

        // Harini obj = new Harini();

        // System.out.println(obj.amount);
        // System.out.println(obj.name);
    }

}
```

```
}  
  
}
```

```
public class ExceptionHandling {  
    public static void main(String[] args) {  
  
        System.out.println("Exception handling");  
  
        int a = 10;  
  
        try {  
  
            int arr[] = {1,2,3};  
            System.out.println(arr[10]);  
            String name = null;  
            System.out.println(name.length());  
            System.out.println(a/0);  
  
        } catch (ArithmeticException e){  
            System.out.println(e.getMessage());  
        } catch (NullPointerException e){  
            System.out.println(e.getMessage());  
        } catch (ArrayIndexOutOfBoundsException e){  
            System.out.println(e.getMessage());  
        } catch (Exception e){  
            System.out.println(e.getMessage());  
        }  
  
        System.out.println("Exception handling over");  
    }  
}
```

```
class Mythread extends Thread{  
    public void run(){  
        System.out.println("Run method");  
    }  
  
    public void display(){  
        System.out.println("Display method");  
    }  
}
```

```

    }
}

// class FIIT extends Thread{

//     public void run(){
//         System.out.println("run method inside FIIT");
//     }
// }

class MyRunnable implements Runnable{

    public void run(){
        System.out.println("Run method inside Interface");
    }

}

public class Multithreading {

    public static void main(String[] args) throws InterruptedException {

        Thread obj1 = new Thread(new MyRunnable());

        obj1.start();
        obj1.join();
        // FIIT obj1 = new FIIT();

        // obj1.start();

        System.out.println("Start");

        Mythread obj = new Mythread();
        obj.display();

        obj.start(); // it create thread

        obj.join();

        // two way => Thread class, Runnable Interface

        System.out.println("End");
    }
}

```

```
}  
  
}
```

Theory

- **Abstraction**

- Achieved using abstract classes and interfaces.
- Abstract class → can have abstract and non-abstract methods.
- Interface → all methods are abstract by default (Java 8+ allows default/static methods).

- **this and super**

- this → refers to current object.
- super → refers to parent class constructor or methods.

- **Interfaces**

- Support multiple inheritance in Java.
- extends → interface to interface.
- implements → class to interface.

- **final keyword**

- final class → cannot be inherited.
- final method → cannot be overridden.
- final variable → constant, cannot be reassigned.

- **Access Modifiers**

- private → within class.
- default → within package.
- protected → within package + subclasses.
- public → accessible everywhere.

- **Exception Handling**

- try-catch-finally → handles runtime errors.
- Multiple catch blocks can handle different exceptions.

- Exception → parent class for all exceptions.
- **Multithreading**
 - Two ways:
 - Extend Thread class.
 - Implement Runnable interface.
 - start() → begins a new thread.
 - join() → waits for a thread to finish.



Coding Round Questions

- Abstract class → Create an abstract class with two abstract methods and one concrete method. Implement it in a child class.
- Interfaces → Demonstrate multiple inheritance using interfaces.
- Final keyword → Show how final prevents overriding and inheritance.
- Access modifiers → Write a program that demonstrates private, default, protected, and public access.
- Exception handling → Write a program that handles ArithmeticException, NullPointerException, and ArrayIndexOutOfBoundsException.
- Multithreading → Create two threads using Thread and Runnable and run them concurrently.



Machine Coding Round Challenge

Problem: Library Management System with OOP Concepts

Requirements:

1. Create an abstract class LibraryItem with abstract methods borrowItem() and returnItem().
2. Implement Book and Magazine classes that extend LibraryItem.
3. Use **interfaces** to add DigitalAccess functionality (e.g., download()).
4. Use **final keyword** to create a LibraryRules class that cannot be inherited.
5. Demonstrate **access modifiers** by keeping bookId private and exposing it via getter/setter.
6. Handle exceptions when a user tries to borrow a non-existent item.
7. Use **multithreading** to simulate multiple users borrowing books at the same time.